

AGENCYOS

AI Agent Memory & Context System

Document 05 of the AgencyOS Master Documentation Set

SHARED MEMORY • PROJECT CONTEXT • CLIENT HISTORY • HANDOFFS • LONG-TERM MEMORY
• CONVERSATION HISTORY

Purpose: Define how AgencyOS stores, retrieves, shares, summarizes, protects and governs the context required for AI agents to operate continuously without losing the history of the client or project.

1. Core Memory Principle

AgencyOS must have one connected memory system, not isolated memories for each AI agent.

Agents are specialized workers, but they must be able to access the relevant shared context of the client, lead, project, scope, approvals, payments, artifacts, conversations and previous agent work.

- The database/workflow state is the authoritative source of business truth.
- AI memory is a context layer that helps agents understand and act on that truth.
- Important facts must be persisted structurally, not left only inside prompts or chat transcripts.
- Agents must never depend on their own private conversation history to understand a project.
- Memory retrieval must be scoped by organization, client, project, role and task permissions.

2. Memory Architecture Overview

RAW EVENTS → STRUCTURED BUSINESS STATE → SUMMARIZED MEMORY → SEMANTIC RETRIEVAL → AGENT CONTEXT → ACTION → NEW EVIDENCE/EVENT

- Raw events preserve what actually happened.
- Structured records represent current business state.
- Summaries compress large histories for efficient context.
- Semantic indexes help find relevant historical information.
- Agent context is assembled dynamically for each task.
- New actions create new events/evidence and update relevant state.
- The memory system must support reconstruction of why an agent made a decision.

3. Memory Layers

Layer	Purpose	Examples
1. Source/Raw History	Preserve original events and evidence	WhatsApp messages, voice-note transcripts, approvals, files
2. Business State	Authoritative current state	Lead status, project status, payment status, milestone
3. Operational Memory	Current working context	Active tasks, blockers, next actions
4. Long-Term Memory	Stable facts worth remembering	Client preferences, recurring requirements, project history
5. Semantic Memory	Find relevant information by meaning	Past project similar to current request
6. Agent Handoff Memory	Transfer work between agents	Inputs, outputs, decisions, blockers
7. Audit Memory	Reconstruct critical actions	Who acted, policy, approval, timestamp, version

4. Client Memory

Purpose

Maintain a persistent understanding of the client across leads, projects, support and future opportunities.

Client memory may contain

- Client identity and contact details.
- Preferred communication language.
- Preferred communication style.
- Business/company information.
- GST/billing preferences.
- Historical projects.

- Completed project types.
- Recurring requirements.
- Important client preferences explicitly provided by the client.
- Known service preferences.
- Approved commercial relationship information.
- Support history.
- Maintenance history.
- Upsell/repeat-project history.

Only information that is useful, legitimate and permitted by policy should be promoted into long-term memory.

5. Lead Memory

- Original lead source, such as Facebook or another configured source.
- First contact date/time.
- Initial message.
- Sales conversation history.
- Requirements discovered so far.
- Questions asked and answered.
- Objections.
- Trust concerns.
- Budget/timeline signals.
- Samples/portfolio/demos already shared.
- Quotes and quote versions.
- Negotiation history.
- Reason for won/lost status.
- Follow-up history.

Lead memory must remain connected to the client record when a lead converts, so the Sales Agent and Project Manager do not lose the context that produced the conversion.

6. Conversation Memory

Raw conversation

- Store the original WhatsApp/email/chat event where permitted.
- Store sender, receiver/channel, timestamp and message identifier.
- Store attachments/voice notes and their references.
- Store transcription for voice notes where available.

Conversation summary

- Maintain rolling summaries so agents do not need the entire conversation every time.
- Summary must identify decisions, requirements, objections, commitments, unresolved questions and next actions.

Important rule

A summary is not the source of truth when an exact original message or structured record exists. Summaries can become stale; authoritative records must win.

7. Project Memory

Project context must include

- Project identity and client.
- Approved quotation.
- Final commercial agreement.
- Approved scope.
- Requirements.
- Feature list.
- Acceptance criteria.
- Milestone/payment plan.
- Payment history.
- UI versions.
- Prototype versions.
- Development tasks.
- Code/repository references.
- QA reports.
- Bug history.
- Change requests.
- Client feedback.
- Client approvals.
- Admin approvals.
- Deployment information.
- Handover information.
- Maintenance/support history.

Every agent working on a project should receive a relevant project-context package rather than starting from zero.

8. Scope Memory

Scope is one of the most important memories in AgencyOS because it determines what the team is actually supposed to build.

- Original requested features.
- Negotiated features.
- Final approved features.
- Explicit exclusions.
- Client-approved additions.
- Paid change requests.
- Free small changes under policy.
- Rejected/out-of-scope requests.
- Scope version and effective date.
- Who approved each scope change.

When a client asks for a new feature, agents must compare it against the latest approved scope before treating it as normal work.

9. Artifact Memory

- UI design files and versions.

- Prototype APK/web builds and versions.
- Specifications.
- Quotes/invoices.
- Client approvals.
- Development artifacts.
- QA reports.
- Screenshots/evidence.
- Deployment/handover packages.
- Artifact metadata should include version, creator/agent, timestamp, status and approval references.

Memory should store references to files/artifacts rather than unnecessarily duplicating large binary content into prompts.

10. Agent Handoff Memory

An agent handoff is a structured transfer of context, not simply 'please continue the work.'

Required handoff payload

- Organization ID.
- Client ID.
- Project ID.
- Lead/opportunity ID where applicable.
- Source agent.
- Destination agent.
- Task ID.
- Task type.
- Current workflow state.
- Approved scope reference/version.
- Relevant artifact/version IDs.
- Task objective.
- Acceptance criteria.
- Client instructions.
- Admin policy/approval references.
- Known decisions.
- Known blockers.
- Previous attempts.
- Failure summary if retrying.
- Expected output.
- Deadline/priority where configured.

11. Example Handoff — Sales → Project Manager

Sales Agent completes conversion and hands off:

- Client identity and contact.
- Full sales conversation summary.
- Requirements and scope.
- Final quotation.

- Negotiated price/payment terms.
- Approved discounts/offers.
- Trust concerns and commitments made.
- Client communication language.
- Client expectations.
- Client approval/acceptance evidence.
- Next required onboarding actions.

The Project Manager should be able to continue the project without asking the client to repeat information already provided during sales.

12. Example Handoff — PM → UI Designer

- Approved scope version.
- Feature list.
- Target users.
- Platforms.
- Required screens/flows.
- Brand/reference assets.
- Client preferences.
- Any Admin design policy.
- Required design themes/variants.
- Acceptance criteria.
- Due/priority information.
- Design version to produce.

13. Example Handoff — UI → Prototype

- Approved UI version.
- Screen/flow inventory.
- Interaction rules.
- Components/design system.
- Assets.
- Approved scope.
- Client feedback already resolved.
- Prototype acceptance criteria.
- Known UI limitations.

14. Example Handoff — Prototype → Developer

- Approved prototype version.
- UI/design reference.
- Functional flow map.
- Approved scope.
- Known prototype limitations that are not production defects.
- Platform requirements.
- Implementation acceptance criteria.

- Client-approved behavior.

15. Example Handoff — Developer → QA

- Build/version.
- Commit/branch/release reference.
- Implemented features.
- Known limitations.
- Test results already run.
- Environment information.
- Database/API changes.
- Relevant deployment configuration.
- Expected behavior.
- Acceptance criteria.
- Known risk areas.

QA must independently verify the implementation. Developer memory is evidence, not proof.

16. Example Handoff — QA → PM/Finance/Handover

- QA report.
- Passed/failed tests.
- Open defects by severity.
- Regression status.
- Production-readiness recommendation.
- Build verification.
- Security/performance findings.
- Required remaining actions.
- Approval recommendation.

Only after required quality gates and payment gates are satisfied can the Project Manager move toward handover/completion.

17. Long-Term Memory Rules

- Promote stable facts, not every transient statement.
- Prefer explicit client statements over inferred preferences.
- Store source/evidence for important memories.
- Store confidence where inference is unavoidable.
- Store created_at and updated_at.
- Support memory expiration/review for facts that can change.
- Allow Admin correction of incorrect memory.
- Never allow an AI hallucination to silently become a permanent client fact.
- When memories conflict, authoritative structured records and recent explicit client statements should be prioritized according to defined policy.

18. Memory Confidence

Suggested memory classes

- EXPLICIT — directly stated by client/Admin/system.
- VERIFIED — confirmed by authoritative business process.
- INFERRED — model-derived and not directly confirmed.
- TEMPORARY — useful only for the current task/stage.
- STALE — previously valid but due for review.
- CONFLICTED — multiple sources disagree.

Only EXPLICIT/VERIFIED information should normally drive important business decisions without additional validation.

19. Memory Retrieval

Agents should receive relevant context dynamically rather than dumping all historical information into every model call.

Retrieval order

- Current task context.
- Current workflow state.
- Latest approved scope and artifacts.
- Recent conversation.
- Relevant client memory.
- Relevant project history.
- Similar historical cases when useful.
- Global business policy/knowledge.

Retrieval should be relevance-ranked and permission-filtered.

20. Context Window Management

- Never send the entire project history by default.
- Use summaries for long conversations.
- Retrieve exact source messages when decisions are ambiguous.
- Retrieve only relevant artifacts.
- Use hierarchical summaries for long-running projects.
- Compress stale history while retaining references to the originals.
- Separate must-know context from nice-to-know context.
- Track context size/cost where provider APIs expose usage.

21. Memory Conflict Resolution

When memory conflicts, the agent must not guess.

- Check authoritative structured state.
- Check latest explicit client/Admin statement.
- Check approval records.
- Check artifact/version status.
- If still unresolved, ask the appropriate human or create an Admin decision request.
- Record the resolution so future agents do not repeat the same ambiguity.

22. Shared Memory vs Agent Private Memory

Memory	Visibility	Purpose
Memory	Shared?	Purpose
Client profile	Yes	Continuity across all agents
Project state	Yes	Single operational truth
Approved scope	Yes	Protect project boundaries
Conversation history	Permissioned shared	Continuity and evidence
Agent task scratchpad	No / task-scoped	Temporary reasoning/context
Agent-specific performance notes	Controlled	Agent improvement/diagnostics
Admin policy	Yes/read-only to agents	Governed authority
Audit history	Permissioned	Accountability/reconstruction

23. Memory Security & Permissions

- Every memory query must carry organization/tenant context.
- Client-facing agents must not retrieve internal notes or private strategy.
- Sales should not automatically access unrelated client projects.
- Agents should access only the minimum data needed for the task.
- Secrets/API keys must never be stored in general memory.
- Payment credentials must remain in secure finance/integration systems.
- Audit records require stronger protection than ordinary conversational summaries.
- Memory deletion/retention must follow the agency's configured legal and privacy requirements.

24. WhatsApp Memory Integration

- Every relevant incoming/outgoing message can create a conversation event.
- Voice notes should be transcribed and linked to the original media reference where supported.
- Important decisions should be extracted into structured records.
- Client approvals should become explicit approval events, not merely summarized text.
- Payment claims should become payment-proof events pending verification.
- Group communication should be linked to project ID.
- Internal Admin-group approvals should be linked to the exact requested action/version.

WhatsApp remains a communication/evidence channel; AgencyOS remains the structured source of truth.

25. Memory for Sales Agent

- Recent conversation summary.
- Client profile.
- Lead source.
- Service catalog.
- Approved pricing/offer policy.
- Relevant previous projects.
- Previous objections.
- Follow-up status.
- Current quotation/version.
- Trust-building context.

Sales Agent should not need to re-ask questions already answered unless the answer is missing, stale or conflicting.

26. Memory for Project Manager Agent

- Project state and timeline.
- Latest scope.
- Milestones/payment status.
- Latest approved UI/prototype.
- Current development/QA state.
- Client approvals/feedback.
- Open blockers.
- Agent task status.
- Project WhatsApp history summary.
- Admin decisions.

PM Agent is the primary consumer of cross-agent operational memory.

27. Memory for UI/Prototype/Developer Agents

- Approved scope.
- Latest approved UI/artifacts.
- Client feedback relevant to the artifact.
- Technical constraints.
- Platform requirements.
- Previous implementation decisions.
- Known defects.
- Acceptance criteria.
- Only relevant project history should be injected into technical tasks.

28. Memory for QA Agent

- Approved acceptance criteria.
- Latest production candidate/build.
- Developer implementation summary.
- Known risk areas.
- Previous defects and regression history.
- Previous QA results.
- Environment information.
- Security/permission requirements.

QA should be able to see enough history to detect regressions without being forced to trust developer summaries.

29. Memory for Finance Agent

- Client billing profile.
- GST preference/status.
- Approved quotation.
- Milestone plan.
- Invoice history.

- Payment proofs.
- Verified payment events.
- Outstanding balances.
- Refund/adjustment records where authorized.

Finance memory must be tied to deterministic financial records. AI summaries must never become the financial source of truth.

30. Memory for Customer Success & Upsell

- Completed project history.
- Maintenance history.
- Support history.
- Client preferences.
- Previous upsell attempts/outcomes.
- Current service catalog.
- Client engagement signals.
- Repeat-project opportunities.
- Commercial restrictions.

Use history to personalize outreach, not to spam the client or invent needs.

31. Memory Lifecycle

CAPTURE → CLASSIFY → VALIDATE → STORE → INDEX → RETRIEVE → USE → UPDATE → REVIEW/EXPIRE

- Capture new event/fact.
- Classify as raw event, structured fact, summary, inferred memory or temporary context.
- Validate when the fact is important.
- Store with source, timestamp and confidence.
- Index for retrieval.
- Retrieve only when relevant.
- Update when new authoritative information arrives.
- Review/expire facts that may become stale.
- Preserve audit evidence where required.

32. Memory Update Rules

- Do not overwrite an important fact without preserving history.
- Prefer versioned records for scope, quotations, approvals, UI, prototypes and key decisions.
- Use append-only events for critical financial/approval/audit transitions where appropriate.
- When a client changes a preference, update the active value and retain the old value in history.
- When a project scope changes, create a new scope version rather than editing history invisibly.

33. Context Packet Standard

Every agent task should receive a generated Context Packet.

- Task objective.

- Agent role.
- Organization/project/client identifiers.
- Current workflow state.
- Relevant policy.
- Relevant recent conversation.
- Relevant structured business facts.
- Approved scope/version.
- Relevant artifact references.
- Previous agent outputs.
- Known blockers.
- Acceptance criteria.
- Required output schema.
- Allowed tools.
- Approval requirements.

34. Context Packet Example — Client asks for a new feature

Before responding, the agent should retrieve:

- Current project.
- Latest approved scope.
- Existing change requests.
- Client's exact message.
- Recent conversation context.
- Current milestone/state.
- Whether the requested feature already exists.
- Whether it is in scope, a small free change or a paid change.
- Relevant commercial policy.

Only then should the agent answer or create a change-request workflow.

35. Preventing Memory Hallucinations

- Never store a model-generated assumption as a verified client fact without provenance.
- Attach source/event IDs to important memories.
- Use confidence/status fields.
- Require confirmation for important inferred preferences.
- Do not allow summaries to invent decisions.
- If the model cannot find evidence, it must say that the fact is unknown.
- When uncertain, retrieve source messages or escalate.

36. Memory Observability

- Track retrieval queries and which memory records influenced a task where feasible.
- Track stale-memory usage.
- Track memory conflicts.
- Track hallucination corrections.
- Track context size/token usage.

- Track retrieval latency.
- Track which memories are frequently used.
- Track failed retrievals and missing context.

This allows the agency to improve prompts, indexing, summaries and agent behavior over time.

37. Acceptance Criteria

- All agents can retrieve permissioned project context.
- Client history survives lead-to-client conversion.
- Project history survives completion.
- WhatsApp conversations can be linked to the correct lead/client/project.
- Important approvals are stored as structured events.
- Payment claims and verified payments are distinct states.
- Scope is versioned.
- UI/prototype/artifacts are versioned.
- Agent handoffs contain structured context.
- Long conversations are summarized without losing source evidence.
- Important memories have provenance.
- Memory access is tenant/role scoped.
- Agents do not receive unnecessary sensitive information.
- Memory conflicts trigger validation rather than guessing.
- Admin can correct important incorrect memory.
- Completed project history remains auditable.

38. Final Architecture

AGENCYOS MEMORY = EVENTS + STRUCTURED STATE + SUMMARIES + SEMANTIC INDEX
+ ARTIFACT REFERENCES + HANDOFFS + AUDIT

The memory system exists to make the AI workforce feel like one experienced internal team. The Sales Agent can hand a client to the Project Manager without losing context. The Project Manager can hand approved UI to developers. Developers can hand builds to QA. QA can hand verified readiness back to the PM. Finance can verify payments against deterministic records. Customer Success can understand the entire relationship after the project is completed.

The rule is simple: remember what matters, prove what matters, retrieve only what is relevant, and never let AI memory replace the actual source of truth.